Confluent Tableflow → Unity Catalog Lineage Integration (POC Design of **LineageBridge**)

Created by Daniel Takabayashi 04/03/2026

1. Background and Objectives

Goal: Design a **customer-shareable POC** that fills Tableflow's current gap around **cross-platform data lineage**, by:

- Discovering **stream lineage** across Confluent Cloud (Kafka, Connect, Flink/ksqlDB, Tableflow, governance).
- Mapping that lineage to **Databricks Unity Catalog** (UC), especially the **Delta tables** that Tableflow publishes.
- Using **only public Confluent Cloud and Databricks APIs**, with clear evolution paths.

We consider three complementary strategies:

1. **UC lineage index in Delta** (store the Confluent lineage graph as Delta tables in UC).
2. **UC BYOL (Bring Your Own Lineage) bridge** (use UC external lineage APIs to push Confluent nodes into the UC Lineage graph).
3. **Centralized lineage hub** (optional higher-end architecture integrating multiple catalogs/platforms via a central lineage system).

This document summarises the research and proposes a concrete POC architecture.

---

2. What Tableflow Provides Today (Lineage-Relevant)

2.1 Role and capabilities

Tableflow is a Confluent Cloud feature that **represents Kafka topics and schemas as open table formats** (Apache Iceberg, Delta Lake) in object storage, automating schematization, type conversions, schema evolution, CDC materialization, catalog publishing, and table maintenance. It uses **Schema Registry** as the schema source of truth: when a topic is enabled for Tableflow, the associated SR schema is used to generate the table schema; topics + schemas are treated as the canonical source of truth.

Tableflow integrates with catalogs including **AWS Glue, Snowflake Open Catalog / Polaris, and Databricks Unity Catalog**.

2.2 Structural mappings (topic → table → catalog)

From a lineage perspective, Tableflow provides strong **structural edges**:

- **Kafka topic → Iceberg/Delta table**: each `TableflowTopic` represents a Kafka topic configured for Tableflow, including environment, Kafka cluster, table format(s), storage config, and status.
- **Tableflow table → external catalog object** via `CatalogIntegration`:
  - For **Unity Catalog**:
    - Tableflow creates a **UC schema named after the Kafka cluster ID**.
    - All topics with Delta format enabled are published as **external Delta tables** under that schema.
  - For **AWS Glue**:
    - Database name = cluster ID; table name = topic name.

This yields deterministic mappings like:

> `env + cluster_id + topic_name` → `Iceberg/Delta table + storage path` → `UC catalog.schema.table`.

2.3 Integration with Stream Governance & Table formats

- Tableflow is tightly coupled to **Schema Registry** and **Stream Governance**:
  - Requires schemas; schemaless topics are not supported.
  - Uses SR compatibility modes; schema incompatibilities can suspend materialization and are exposed via metrics.
- Uses **Flink's deserializers** and type system; if a schema works with Flink, it typically works with Tableflow.
- With **Iceberg V3**, Tableflow can leverage **row-level lineage** and semi-structured Variant type, stored within Iceberg metadata.

2.4 Current lineage limitations

However:

- Tableflow **does not natively export Confluent Stream Lineage into Unity Catalog**; UC lineage remains based on Databricks runtime and BYOL APIs.
- Public Tableflow APIs are focused on **table configuration & catalog sync**, not full streaming topology; edges between topics, connectors, Flink, ksqlDB, etc. must be obtained from **other Confluent services** (Stream Governance, Connect, Flink, ksqlDB APIs).

So Tableflow should be treated as **the authoritative mapping between Kafka topics and tables + UC tables**, not as the primary lineage engine.

---

3. Confluent Cloud Lineage Signals (Public APIs)

A POC must rely only on **public APIs** and surfaces:

3.1 Kafka Admin (Topics, Consumer Groups, ACLs)

- **Kafka REST Admin v3**:
  - Topics: list, describe, configs, partitions, retention.
  - Consumer groups: list and describe groups/members and lag.
  - ACLs: who can read/write which topics/groups.

Signals: Topic inventory, consumer-group→topic relationships (runtime edges), and access control edges.

3.2 Stream Catalog (Stream Governance)

- **Catalog REST APIs (Entity/Search/Types)**:
  - Entities: topics, schemas, schema fields, connectors, clusters, environments, Flink compute pools, streaming pipelines.
  - Business metadata: tags, classifications, owners.

Signals: Semantic graph (topics↔schemas↔connectors↔pipelines) with business metadata (data products, owners, PII).

3.3 Schema Registry

- **Schema Registry REST APIs**:
  - Subjects, schema versions, references, compatibility.

Signals: Schema evolution history and reuse across topics; important for column-level semantics and contracts.

3.4 Connect API

- **Confluent Cloud Connect API**:
  - List connectors; retrieve configs; plugin metadata.

Signals: Source/sink connectors' configs show:

- Source: external system/table → Kafka topic(s).
- Sink: Kafka topic(s) → external system/table (including Delta/objects).

3.5 ksqlDB

- **ksqlDB cluster API** to list clusters and endpoints.
- **ksqlDB data-plane REST** via `http_endpoint`, to run `SHOW QUERIES`, `SHOW STREAMS/TABLES`, `DESCRIBE EXTENDED`, etc.

Signals: For each persistent query:

- Input topics (`FROM`, `JOIN`).
- Output topics (`INTO`, `INSERT INTO`).

Edges: `input_topic(s)` → `ksqldb_query` → `output_topic(s)`.

3.6 Flink on Confluent Cloud

- **Compute pool APIs** (`fcpm/v2`) – enumerate compute pools.
- **Statement / Table API** – submit/list Flink SQL statements via REST `https://flink.{region}.{cloud}.confluent.cloud`.

Signals: From Flink SQL DDL/DML:

- Kafka source and sink tables (connector `kafka`, `topic` option).
- Transformations between topics/tables.

Edges: `source_topic/table` → `flink_job` → `sink_topic/table`.

3.7 Metrics API & Audit Logs

- **Metrics API**: topic throughput, consumer lag, connector/Flink metrics.
- **Audit logs** (`confluent-audit-log-events` topic):
  - Authn/authz events, admin actions (create/delete topics, connectors, ksqlDB clusters, Flink pools, etc.).

Signals: Activity and performance of edges, and lifecycle events for nodes/edges.

3.8 Tableflow APIs

- **`TableflowTopic` & `CatalogIntegration`**:
  - Topic-level Tableflow configuration and catalog sync metadata.

Signals: Deterministic:

- `kafka_topic` → `tableflow_table` → `uc_table`
  with cluster ID and topic name mapping to UC schema and table.

---

4. Unity Catalog Lineage and BYOL

4.1 Native UC lineage

Unity Catalog automatically tracks:

- Runtime lineage for **tables/views/columns** across queries, notebooks, jobs, dashboards, Lakehouse pipelines.
- Stores this in system tables (`system.access.table_lineage`, `system.access.column_lineage`) with ~1 year retention.

This covers **everything inside Databricks** after data is in UC.

4.2 BYOL (Bring Your Own Lineage)

UC offers a **Public Preview** external lineage feature:

- **External Metadata** securables:
  - Represent external entities (e.g., PostgreSQL table, Kafka topic, Tableau dashboard) with fields like `name`, `system_type`, `entity_type`, optional `url`, `columns`, and `properties` JSON.
- **External Lineage relationships**:
  - Connect an external metadata object to:
    - A UC table, model version, path, or another external metadata object.
  - Direction: **upstream** or **downstream** relative to the external node.
  - Optional column mappings and JSON properties for extra context.

Important constraints:

- Feature is **Public Preview** with quotas: up to 10k external metadata objects and 100k external lineage relationships per metastore.
- **External lineage is not stored in UC system tables**; it's only retrievable via the external lineage REST API and visible in the UI.
- Databricks does **not** ship built-in crawlers/connectors; customers must build their own integration.

For Confluent, this is the **official way** to show Kafka/Connect/Flink/Tableflow nodes inside the **UC Lineage graph**.

---

5. Implementation Options

5.1 Option 1 – UC Lineage Index in Delta (Core POC)

Idea: Build the Confluent lineage graph using public APIs and persist it as **Delta tables** (nodes + edges) in a UC-governed catalog.

Components:

- **Lineage Extractor**:
  - Calls Confluent APIs (Kafka, Connect, SR, Catalog, ksqlDB, Flink, Tableflow).
  - Normalizes entities into:
    - `lineage_nodes(node_id, system, type, qualified_name, attributes_json, tags_json, first_seen_ts, last_seen_ts)`
    - `lineage_edges(src_id, dst_id, edge_type, confidence, attributes_json, first_seen_ts, last_seen_ts)`
- **UC Sync job**:
  - Writes/Upserts into Delta tables in a UC schema (`governance.lineage_*`).
- **Consumer layer**:
  - Databricks SQL / Lakeview dashboards & notebooks to visualize:
    - Upstream Kafka/Confluent lineage for any UC table.
    - Downstream UC usage for any Kafka topic.

Pros:

- Uses only **public APIs** on both sides.
- Simple to explain and operate; no dependency on UC previews.
- Graph is **queryable and exportable** (can feed Purview, DataHub, Collibra, etc.).
- Naturally extensible to include UC internal lineage (from system tables).

Cons:

- Lineage appears in **custom dashboards**, not in the UC Lineage tab.
- Freshness is batch (unless extended with streaming ingestion).

5.2 Option 2 – UC BYOL External-Lineage Bridge

Idea: Reuse the same Confluent lineage graph but additionally push **selected Confluent entities into the UC Lineage graph** using BYOL.

Mapping:

- For each Kafka topic that backs a UC table (via Tableflow):
  - Create an **External Metadata** object:
    - `system_type = Kafka`, `entity_type = topic`, name like `confluent_topic_lkc-xyz_orders_enriched`, `url` to Confluent topic page, `properties` with `cluster_id`, `environment_id`, etc.
  - Create an **External Lineage** relationship:
    - External Kafka topic **upstream of** UC table (`object_type = TABLE`, `object_name = catalog.schema.table`).
- Optionally also:
  - External metadata for connectors, Flink/ksqlDB jobs, upstream DB tables.
  - Chain them in UC's graph: `db_table` → `connector` → `kafka_topic` → `uc_table`.

Pros:

- Confluent entities appear **inside the native UC Lineage graph**, which is where Databricks users live.
- You can deep-link from UC nodes straight back into Confluent via the external metadata `url`.
- BYOL is the blessed UC mechanism for including non-Databricks lineage.

Cons:

- Depends on a **Public Preview** feature (BYOL), with quotas and evolving APIs.
- External lineage is invisible to tools that read only `system.access.*` system tables.
- More permissions to manage (CREATE EXTERNAL METADATA, MODIFY, SELECT/MODIFY on tables).

Best use: As an **add-on** to Option 1, for a curated subset of "gold" tables where UC Lineage UX is especially valuable.

5.3 Option 3 – Centralized Lineage Platform (Hub)

Idea: Instead of privileging UC, use a **central lineage platform** (DataHub, OpenMetadata, Collibra, Atlan, Marquez/OpenLineage) as the **system of record** and UI. Confluent and UC both feed it.

Pattern:

- **Confluent → Hub**:
  - Either via OpenLineage events (from connectors/Flink/ksqlDB/Tableflow) or via the same Extractor writing into the hub's model.
- **UC → Hub**:
  - Via UC lineage system tables and/or OpenLineage Spark listener on Databricks clusters.
- **Hub UI**:
  - Shows end-to-end lineage across many systems (Confluent, Databricks, Snowflake, dbt, BI tools, etc.).
- Optional: **write-back** to UC via BYOL (and to other catalogs) for local UX.

Pros:

- Truly **cross-platform** and vendor-neutral.
- Fits customers already investing in DataHub/Collibra/Atlan/OpenMetadata/Purview.
- Can serve as a single lineage API for multiple consumers.

Cons:

- Heavier for a POC; requires running another platform.
- Overlaps with UC's governance role; may be politically or operationally complex.
- More moving parts and identity/RBAC alignment.

Best use: For customers who already have or clearly want a **central governance platform**. For a generic Confluent-shareable POC, it's better described as an **advanced option** built on the same extractor.

---

6. Recommended POC Architecture

For a shareable, realistic POC that can evolve, the recommendation is:

> **Core:** Option 1 – UC Lineage Index in Delta
> **Enhancement:** Option 2 – UC BYOL bridge for selected tables
> **Advanced:** Option 3 – optional integration with a central lineage hub

6.1 Scope of the initial POC

In scope:

- A small but representative subset of Confluent workloads:
  - 1–2 Kafka clusters with Stream Governance Advanced.
  - 2–5 key connectors (source + sink).
  - A few ksqlDB/Flink jobs.
  - Tableflow + Unity Catalog integration for those clusters.
- Build the lineage graph for:
  - `external_system` ↔ `connector` ↔ `topic(s)` ↔ `ksqlDB/Flink` ↔ `topic(s)` ↔ `tableflow_table` ↔ `uc_table`.

Out of scope for first iteration (but design should allow later):

- Row-level lineage (Iceberg V3).
- Full audit-log-driven temporal lineage.
- Integration with multiple catalogs (Purview, Collibra, etc.).

6.2 Components

1. **Confluent Lineage Extractor (service or job)**

   Responsibilities:

   - Enumerate:
     - Environments & clusters.
     - Topics & consumer groups.
     - Connectors & configs.
     - ksqlDB clusters & persistent queries.
     - Flink compute pools & SQL jobs.
     - Tableflow topics & catalog integrations.
   - Join with:
     - Schema Registry (subjects, schemas).
     - Stream Catalog (entities, tags).
   - Build **in-memory lineage graph** (nodes + edges).
   - Emit:
     - Node and edge batches to Databricks (Delta) for Option 1.
     - Optional BYOL payloads for Option 2.
     - Optional OpenLineage or hub-specific payloads for Option 3.

2. **Databricks Lineage Loader**

   - Receives batches from the Extractor (via JDBC, file drop, or HTTP).
   - Upserts into **Delta tables**:
     - `governance.lineage_nodes`

        - `governance.lineage_edges`
    - Creates a few **helper views**:
        - `v_uc_table_upstream_confluent`
        - `v_kafka_topic_downstream_uc`

3. **UC BYOL Writer (optional module)**

    - For selected “gold” UC tables (e.g., those in a given catalog or schema):
        - Finds their upstream Kafka topics and connectors from the graph.
        - Ensures external metadata objects exist for those topics/connectors.
        - Creates external lineage relationships from those external metadata objects to UC tables using BYOL APIs.

4. **Demo assets**

    - **Databricks notebook(s):**
        - Show lineage for a given UC table.
        - Show all UC tables fed by a given topic or data product.
    - **One or two Lakeview/BI dashboards**:
        - “Kafka → UC lineage” visual.
    - Optional **README** and diagrams.

6.3 Data model (summary)

Nodes:

- `kafka_topic`, `connector`, `ksqldb_query`, `flink_job`, `tableflow_table`, `uc_table`, `schema`, `external_dataset`, `consumer_group`.

Edges:

- `connector` ↔ `kafka_topic` (source/sink).
- `ksqldb_query` / `flink_job` between input and output topics.
- `kafka_topic` → `tableflow_table` → `uc_table`.
- Optionally UC → UC edges via UC system lineage tables.

The same model can be reused to:

- Generate BYOL external metadata + relationships.
- Generate OpenLineage events for a central hub.

---

7. Evolution Paths

Once the basic POC is working, you can evolve it in several directions:

1. **Improve freshness and temporal fidelity**

    - Move from periodic full scans to:
        - Change-based updates via audit logs and Metrics API.
        - Or streaming lineage events via a Kafka topic and DLT.

2. **Deepen Databricks integration**

    - Ingest UC system lineage tables to join Confluent → UC → downstream jobs/dashboards in one query.
    - Use BYOL more extensively to place external Confluent entities in UC Lineage.

3. **Add a central lineage hub**

    - Feed DataHub/OpenMetadata/Collibra/Marquez with:
        - The same node/edge graph.
        - Optionally OpenLineage events.
    - Use the hub as the primary cross-system lineage UI and API, with UC BYOL as a "local view."

4. **Prepare for product evolution**

    - As Confluent adds:
        - Richer Stream Lineage APIs.
        - Unified catalog/lineage APIs.
    - And as Databricks evolves BYOL and lineage, you can gradually:
        - Replace some custom extraction with native exports.
        - Tighten the integration between Confluent lineage and UC system tables.

---

8. Summary

- **Tableflow** already solves the "topics as tables" and "tables into Unity Catalog" problems, but doesn't push **Confluent Stream Lineage** into UC today.
- **Confluent Cloud** exposes enough public APIs (Kafka/Connect/SR/Catalog/ksqlDB/Flink/Tableflow/Metrics/Audit Logs) to reconstruct a rich **stream lineage graph** per environment.

- **Unity Catalog** provides:
  - Native runtime lineage for UC-managed tables/jobs.
  - A **BYOL API** to add external nodes and edges to its lineage graph.
- A **practical, shareable POC** is to:
  - Build a **Confluent lineage extractor** and store the resulting graph as **Delta tables in UC** (Option 1).
  - Optionally **project curated Confluent entities into UC's Lineage graph** using BYOL for high-value tables (Option 2).
  - Keep the design extensible so that **central lineage hubs** (DataHub/Collibra/OpenMetadata) can be plugged in later (Option 3).

This gives customers immediate, concrete value today while aligning with both Confluent and Databricks' published roadmaps and public interfaces.

Sources

- Tableflow in Confluent Cloud | Confluent Documentation
- Tableflow in Confluent Cloud | Confluent Documentation
- Supported Types with Tableflow in Confluent Cloud | Confluent Documentation
- Integrate Catalogs with Tableflow in Confluent Cloud | Confluent Documentation
- Integrate Catalogs with Tableflow in Confluent Cloud | Confluent Documentation
- tableflow.schemas.ts
- Integrate Tableflow with Unity Catalog in Confluent Cloud | Confluent Documentation
- Delta Lake & Unity Catalog Now Integrate With Tableflow
- Integrate Tableflow with the AWS Glue Catalog in Confluent Cloud | Confluent Documentation
- AWS Glue Data Catalog
- Tableflow L200 Content
- Tableflow L200 Content
- Iceberg V3- Variant Data type - PRFAQ
- Thread between Daniel, Glean, and Kasun
- NSTech - Architecture Discussion & Recomendation
- Stream Lineage 1.5
- Stream Lineage V2 Roadmap
- Confluent Cloud API Reference Documentation
- Use access control lists (ACLs) for Confluent Cloud | Confluent Documentation
- Kafka REST APIs for Confluent Cloud | Confluent Documentation
- Stream Catalog on Confluent Cloud | Confluent Documentation
- Stream Catalog REST API Usage for Confluent Cloud | Confluent Documentation
- Schema Registry API Reference | Confluent Documentation
- Endpoint Formats for Confluent Cloud

- Quick Start for Schema Management on Confluent Cloud | Confluent Documentation
- Confluent Cloud API for Connect Usage Examples | Confluent Documentation
- Quick Start for ksqlDB Cluster API on Confluent Cloud | Confluent Documentation
- Manage Flink Compute Pools in Confluent Cloud for Apache Flink | Confluent Documentation
- Table API on Confluent Cloud for Apache Flink | Confluent Documentation
- Confluent Cloud Metrics | Confluent Documentation
- Thread between Anusha and Gary
- Confluent Cloud audit log concepts | Confluent Documentation
- confluent tableflow catalog-integration | Confluent Documentation
- [APIE-650] Tableflow Catalog integration support for Unity Catalog
- Tableflow in Confluent Cloud | Confluent Documentation
- View data lineage using Unity Catalog | Databricks on AWS
- View data lineage using Unity Catalog - Azure Databricks | Microsoft Learn
- Databricks Data Lineage: Complete Tracking & Management Guide
- Supported transformation details
- Bring your own data lineage | Databricks on AWS
- Bring your own data lineage - Azure Databricks | Microsoft Learn
- Bring your own data lineage - Azure Databricks | Azure Docs
- Lineage Intern Project
- Using OpenLineage with Spark | OpenLineage
- Native data lineage in Debezium with OpenLineage
- test_databricks_kafka_lineage.py
- metadata.py
- Supported transformation details
- How to Add External Data Sources to Unity Catalog Lineage: A Complete Guide for Kafka and IoT Integration — SunnyData
- Confluent Tableflow + Databricks Unity Catalog | Real-Time Data to AI-Ready Tables

Written with Glean Assistant